NumPy Functions

In [ ]:
```
import numpy as np
d = np.array([100,23,45,67,89,98,87,65,43,32,56,78])
ar = np.array([[12,3,6],[4,335,64]])
arr = np.array([12,3,6,4,33,64,54,82,84,2,98,87])
```

1. `np.array()` — Creating arrays

Description: Converts a Python list (or nested list) into a NumPy `ndarray` — the basic building block for everything else.

In [ ]:
```
a1 = np.array([1, 2, 3, 4])
print(a1)
```

```
[1 2 3 4]
```

In [ ]:
```
a2 = np.array([[12,3,6], [4,335,64]])
print(a2)
```

```
[[ 12   3   6]
 [  4 335  64]]
```

In [ ]:
```
a3 = np.array([1, 2, 3], dtype='float64')   # array([1., 2., 3.])
print(a3)
print(a3.dtype)
```

```
[1. 2. 3.]
float64
```

2. `arr.size` / `arr.ndim` / `arr.shape` — Array properties

Description: `.size` = total number of elements, `.ndim` = number of dimensions (axes), `.shape` = tuple giving size along each axis. In your notebook, `ar` had size 6 and ndim 2.

In [ ]:
```
print(ar.size)
print(ar.ndim)
print(ar.shape)
```

```
6
2
(2, 3)
```

In [ ]:
```
print(len(ar) * len(ar[0]))
print(np.array(ar).ndim == 2)
```

```
6
True
```

In [ ]:
```
rows, cols = ar.shape
print(f"{rows}x{cols} matrix, {ar.size} items")
```

```
2x3 matrix, 6 items
```

3. `reshape()` — Changing array dimensions

Description: Rearranges the same data into a new shape without changing values. `-1` lets NumPy auto-calculate one dimension (your note: "*agar pata na ho kitne rows banane hai toh -1*"). `reshape` returns a **view**, so changes reflect in the original.

In [ ]:
```
k = arr.reshape(3, 4)
print(k)
```

```
[[12  3  6  4]
 [33 64 54 82]
 [84  2 98 87]]
```

In [ ]:
```
k2 = arr.reshape(-1, 3)
print(k2)
```

```
[[12  3  6]
 [ 4 33 64]
 [54 82 84]
 [ 2 98 87]]
```

In [ ]:
```
k3 = arr.reshape(2, 2, 3)
print(k3)
```

```
[[[12  3  6]
  [ 4 33 64]]

 [[54 82 84]
  [ 2 98 87]]]
```

In [ ]:
```
v = arr.reshape(3, 4)
v[0, 0] = 100
print("original arr after edit:", arr)
```

```
original arr after edit: [100   3   6   4  33  64  54  82  84   2  98  87]
```

4. `sum()` with axis — Aggregation

Description: Adds up elements. No axis → sum of the whole array. `axis=0` collapses rows (adds down columns / "*vertically*"), `axis=1` collapses columns (adds across each row / "*horizontally*").

In [ ]:
```
print(ar.sum())
print(ar.sum(axis=0))
```

```
print(ar.sum(axis=1))
```

```
424
[ 16 338  70]
[ 21 403]
```

In [ ]:
```
np.sum(ar, axis=1)
```

Out[ ]:
```
array([ 21, 403])
```